FAIRHAVEN HEALTH

FairHaven Partner API · v1

Руководство для разработчиков партнёра: каталог, остатки, заказы и уведомления о статусах

Базовый адрес	`https://api.fairhaven.uz/partner/v1`
Спецификация	`https://api.fairhaven.uz/partner/openapi.yaml`
Формат	HTTPS + JSON, ключ в заголовке `Authorization: Bearer`
Версия	1.0 · ноябрь 2026

Конфиденциально — для партнёров FairHaven

Руководство для разработчиков партнёра: как подключиться к API аптеки FairHaven Health, получать каталог и остатки, отправлять заказы и узнавать решение аптеки.

Это обычный HTTPS + JSON. Примеры — на `curl`, Python и Node.js, но подойдёт любой язык. Готовый модуль на Python со всеми методами — `partner_client.py`. Машиночитаемый контракт (OpenAPI 3) — `partner-api.openapi.yaml`, он же доступен по адресу `https://api.fairhaven.uz/partner/openapi.yaml` (импортируется в Postman, Insomnia или Swagger Editor).

Содержание

1 Что умеет API

2 Что понадобится

3 Режимы подключения

4 Быстрый старт (5 минут)

5 Общие правила

6 Каталог

7 Остатки

8 Лимиты

9 Заказы

10 Ошибки

11 Webhooks — уведомления о смене статуса

12 Без webhook: опрос

13 Тестовый режим: что проверить

14 Переход в боевой режим

15 Частые вопросы

1. Что умеет API

Задача	Метод
Проверить ключ и режим подключения	`GET /me`
Каталог: названия, бренды, штрихкоды, ИКПУ, фото, цены	`GET /products`, `GET /products/{id}`
Остатки и цены всех товаров	`GET /stock`
Отправить заказ	`POST /orders`
Узнать статус заказа	`GET /orders/{order_id}`, `GET /orders`
Отменить заказ	`POST /orders/{order_id}/cancel`
Подтвердить, что заказ доставлен и оплачен	`POST /orders/{order_id}/complete`
Проверить свой webhook	`POST /webhook/test`
Получать смену статуса без опроса	webhook `order.status_changed`

Как устроена работа в двух словах: вы показываете своим покупателям наш каталог с нашими остатками, отправляете нам заказ, аптека принимает или отклоняет его в течение нескольких минут, принятый заказ резервируется, вы доставляете его покупателю и сообщаете нам, что заказ доставлен и оплачен — в этот момент продажа проводится в нашем учёте.

2. Что понадобится

Что	Значение
Базовый адрес	`https://api.fairhaven.uz/partner/v1`
Токен чтения	`fhp_t_…` — выдаётся при подключении
Секрет заказов	`fhp_s_…` — выдаётся при подключении
Секрет подписи webhook	`whsec_…` — выдаётся, когда вы сообщите адрес webhook
Ваш адрес webhook (необязательно)	`https://…` — сообщаете нам вы

Ключи присылаются отдельным сообщением, не вместе с этим документом. Храните их в переменных окружения или в хранилище секретов — не в коде, не в репозитории и не в логах. Если ключ мог утечь, напишите нам: мы выпустим новую пару, старая перестанет работать сразу.

Как передавать ключ

В заголовке `Authorization`:

```
Authorization: Bearer fhp_t_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

или, если вашему HTTP-клиенту так удобнее, в заголовке `X-Api-Key`:

```
X-Api-Key: fhp_t_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Ключ в адресе (`?token=`) не принимается.

Два ключа — две задачи

Ключ	Что можно
Токен чтения `fhp_t_…`	`/me`, каталог, остатки, просмотр заказов
Секрет заказов `fhp_s_…`	всё то же **плюс** создание, отмена и завершение заказов, `/webhook/test`

Токен чтения можно спокойно отдать сервису, который только синхронизирует витрину. Секрет заказов нужен только той части вашей системы, которая оформляет заказы. Если на метод заказов отправить токен чтения, ответ будет `401 unauthorized` с понятным текстом: `this endpoint needs the order secret (fhp_s_…); the read token was sent`.

3. Режимы подключения

Режим	Что происходит
`test`	Работает весь API. Каталог и остатки — настоящие. Заказы проходят все статусы, но **не трогают** реальный склад и учёт, аптека их не видит. Решение по заказу принимает робот примерно через 10–15 секунд.
`live`	Боевой режим: заказы получает оператор аптеки, товар резервируется, после `complete` проводится продажа.
`off`	Доступ выключен: любой запрос отвечает `403 partner_disabled`.

Каждый партнёр начинает в режиме `test`. Когда вы прошли тестовые сценарии (раздел 12), мы вместе проверяем интеграцию и переключаем вас в `live`. Ключи при этом **не меняются**. Текущий режим всегда виден в `GET /me` → `partner.mode`, а у каждого заказа есть поле `test: true/false`.

4. Быстрый старт (5 минут)

Положите ключи в переменные окружения:

```bash
export FAIRHAVEN_TOKEN='fhp_t_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX'
export FAIRHAVEN_SECRET='fhp_s_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX'
export FH='https://api.fairhaven.uz/partner/v1'
```

Шаг 1. Проверьте ключ.

```bash
curl -s "$FH/me" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

```json
{
  "partner": { "code": "demo-partner", "name": "Demo Partner", "mode": "test" },
  "key": "token",
  "shop": { "name": "FAIRHAVEN HEALTH" },
  "webhook_configured": false,
  "server_time": "2026-11-12T08:00:00.000Z"
}
```

Шаг 2. Возьмите остатки.

```bash
curl -s "$FH/stock?limit=500" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

```json
{
  "items": [
    { "product_id": 701, "quantity": 12, "available": true, "price": 145000 },
    { "product_id": 702, "quantity": 0, "available": false, "price": 89000 }
  ],
  "total": 2, "limit": 500, "offset": 0,
  "generated_at": "2026-11-12T08:00:01.000Z"
}
```

Шаг 3. Отправьте тестовый заказ.

```bash
curl -s -X POST "$FH/orders" \
  -H "Authorization: Bearer $FAIRHAVEN_SECRET" \
  -H "Content-Type: application/json" \
  -d '{
        "order_id": "TEST-0001",
        "items": [ { "product_id": 701, "quantity": 2 } ],
        "customer": { "name": "Тестовый покупатель", "phone": "+998900000000" },
        "delivery": { "type": "courier", "address": "Ташкент, тестовый адрес, 1" }
      }'
```

Ответ `201` — заказ в статусе `new`, в поле `decision_deadline` — до какого времени аптека ответит.

Шаг 4. Узнайте решение (через 10–15 секунд в тестовом режиме):

```bash
curl -s "$FH/orders/TEST-0001" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

Статус `accepted` — товар зарезервирован, можно доставлять.

Шаг 5. Подтвердите доставку и оплату.

```bash
curl -s -X POST "$FH/orders/TEST-0001/complete" \
  -H "Authorization: Bearer $FAIRHAVEN_SECRET"
```

Статус `completed` — цикл пройден.

5. Общие правила

- **Формат.** Запросы и ответы — JSON в UTF-8. Для `POST` с телом указывайте `Content-Type: application/json`. Тело больше 256 КБ отклоняется с кодом `413` (такой ответ может прийти не в JSON — проверяйте код статуса).
- **Деньги** — целое число сумов (UZS), без тийинов и без дробной части: `145000` значит 145 000 сум.
- **Количество** — целые штуки (`unit: "pcs"`).
- **Время** — ISO 8601 в UTC, например `2026-11-12T08:00:00.000Z`. Ташкент — UTC+5.
- **Постраничный вывод** — у `/products`, `/stock`, `/orders`: параметры `limit` (1–500, по умолчанию 100) и `offset` (с 0). Ответ: `{"items": [...], "total": N, "limit": L, "offset": O}`. Чтобы забрать всё, увеличивайте `offset` на `limit`, пока `offset < total`.
- **Ошибки** всегда в одном виде (исключение — `429`, см. раздел 8):

```json
{
  "error": {
    "code": "insufficient_stock",
    "message": "not enough stock for some items",
    "details": { "items": [ { "product_id": 701, "requested": 5, "available": 3 } ] }
  }
}
```

Решения в вашем коде принимайте по `error.code` (он стабилен), `message` — пояснение для человека, `details` есть не всегда.

6. Каталог

`GET /products` — список товаров

Параметры: `limit`, `offset`, `search` (поиск по названию, бренду, артикулу или штрихкоду, до 100 символов, без учёта регистра). Сортировка — по названию.

```bash
curl -s "$FH/products?limit=100&offset=0" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
curl -s "$FH/products?search=4600000000000" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

Один товар:

```json
{
  "id": 701,
  "name": "Витамин D3 2000 МЕ, 60 капсул",
  "name_uz": "D3 vitamini 2000 XB, 60 kapsula",
  "brand": "Example Brand",
  "category": "vitamins",
  "category_name": "Витамины",
  "barcode": "4600000000000",
  "sku": "EX-D3-60",
  "description": "…",
  "description_uz": "…",
  "images": ["https://api.fairhaven.uz/…/701-1.jpg"],
  "mxik": "00000000000000000",
  "package_code": "0000000",
  "unit": "pcs",
  "price": 145000,
  "old_price": 160000,
  "quantity": 12,
  "available": true,
  "updated_at": "2026-11-12T07:55:00.000Z"
}
```

Поле	Что это
`id`	Номер товара у нас. Используйте его в заказах (`product_id`).
`name`, `name_uz`	Название на русском и узбекском (узбекского может не быть — пустая строка).
`brand`, `category`, `category_name`	Бренд и категория. Коды категорий: `cosmetics`, `parapharmaceuticals`, `supplements`, `vitamins`, `hygiene`, `drinks`.
`barcode`, `sku`	Штрихкод и артикул.
`description`, `description_uz`	Описание (может быть пустым).
`images`	Ссылки на фото, первое — главное.
`mxik`, `package_code`	ИКПУ и код упаковки для фискального чека.
`price`	Цена за штуку в сумах — по ней будет посчитан заказ.
`old_price`	Цена до скидки, только если она больше `price`, иначе `null`.
`quantity`, `available`	Сколько можно заказать прямо сейчас (как в `/stock`).
`updated_at`	Когда карточка менялась последний раз.

`GET /products/{id}` — один товар

```bash
curl -s "$FH/products/701" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

Если товара больше нет в каталоге — `404 product_not_found`.

Как часто обновлять каталог. Карточки меняются редко: достаточно полной выгрузки раз в час или раз в сутки. Цены и количество берите из `/stock`.

7. Остатки

`GET /stock` — остатки и цены всех товаров каталога

```bash
curl -s "$FH/stock?limit=500&offset=0" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

```json
{
  "items": [
    { "product_id": 701, "quantity": 12, "available": true,  "price": 145000 },
    { "product_id": 702, "quantity": 0,  "available": false, "price": 89000 }
  ],
  "total": 2, "limit": 500, "offset": 0,
  "generated_at": "2026-11-12T08:00:01.000Z"
}
```

Важно:

- В ответе **все** товары каталога, в том числе закончившиеся — с `quantity: 0` и `available: false`. Товар, который пропал из списка, **убран из каталога**: снимите его с вашей витрины.
- `quantity` — сколько можно продать именно вам: остаток в аптеке минус заказы других каналов продаж минус страховой запас для торгового зала. Заказывать больше этого числа нельзя.
- `price` может меняться — всегда показывайте покупателю цену из последнего ответа.
- Рекомендуемая частота опроса — раз в 1–5 минут (одна страница на 500 товаров обычно покрывает весь каталог).
- Остаток меняется между запросами. Окончательную проверку делает `POST /orders`: если товара уже не хватает, вы получите `409 insufficient_stock` с точными числами (раздел 9).

8. Лимиты

Что	Лимит
Запросов на один ключ	240 в минуту
Отклонённых запросов (неверный ключ и т. п.) с одного IP	300 за 5 минут
Строк в заказе	до 50
Количество в строке	1–999
`order_id`	до 64 печатных ASCII-символов без пробелов
Размер тела запроса	256 КБ
`limit` в списках	1–500

При превышении лимита запросов ответ — `429` с телом `{"detail": "Too many requests — slow down and retry"}` (не в общем формате ошибок) и заголовком `Retry-After` — через сколько секунд можно

продолжать. Текущий расход виден в заголовках `RateLimit` и `RateLimit-Policy` каждого ответа. При опросе по рекомендациям этого документа лимит не достигается.

Кроме лимита на ключ есть общий лимит на IP-адрес: 300 запросов в минуту на все ваши ключи вместе (кратковременный всплеск — до 60). При его превышении `429` приходит с телом `{"error": "rate_limited"}` и без `Retry-After` — подождите 1–5 секунд и снизьте частоту.

9. Заказы

9.1 Создание: `POST /orders` (секрет заказов)

```bash
curl -s -X POST "$FH/orders" \
  -H "Authorization: Bearer $FAIRHAVEN_SECRET" \
  -H "Content-Type: application/json" \
  -d '{
        "order_id": "A-100045",
        "items": [
          { "product_id": 701, "quantity": 2 },
          { "product_id": 705, "quantity": 1 }
        ],
        "customer": { "name": "Тестовый покупатель", "phone": "+998900000000" },
        "delivery": { "type": "courier", "address": "Ташкент, тестовый адрес, 1", "comment":
"Домофон 00" },
        "comment": "Позвонить за 15 минут"
      }'
```

Поле	Обязательно	Что это
`order_id`	да	**Ваш** уникальный номер заказа: до 64 печатных ASCII-символов без пробелов. По нему вы потом получаете, отменяете и завершаете заказ.
`items[]`	да	1–50 строк `{product_id, quantity}`; `quantity` — 1–999. Две строки с одним товаром складываются в одну.
`customer.name`	нет	Имя покупателя, до 100 символов.
`customer.phone`	нет	Телефон, до 32 символов.
`delivery.type`	нет	`courier` (по умолчанию) или `pickup` (самовывоз из аптеки).
`delivery.address`	нет	Адрес, до 300 символов.
`delivery.comment`	нет	Комментарий к доставке, до 500 символов.
`comment`	нет	Комментарий к заказу, до 500 символов.
`test_outcome`	нет	Только в режиме `test`: `"reject"` — робот отклонит заказ. Любое другое значение или отсутствие — примет. В режиме `live` игнорируется.

Цены в запросе не передаются и не принимаются: заказ всегда считается по нашему каталогу. Цена и сумма каждой строки возвращаются в ответе — сверьте их со своими.

Ответы:

Код	Когда
`201`	Новый заказ создан, статус `new`.
`200`	Заказ с таким `order_id` у вас уже есть — возвращён он, ничего не изменено.
`404 product_not_found`	Товара нет в каталоге; в `details.product_id` — какого.
`409 insufficient_stock`	Не хватает остатка; в `details.items` — что, сколько просили и сколько есть.
`422 validation_error`	Ошибка в данных запроса; в `message` — что именно.

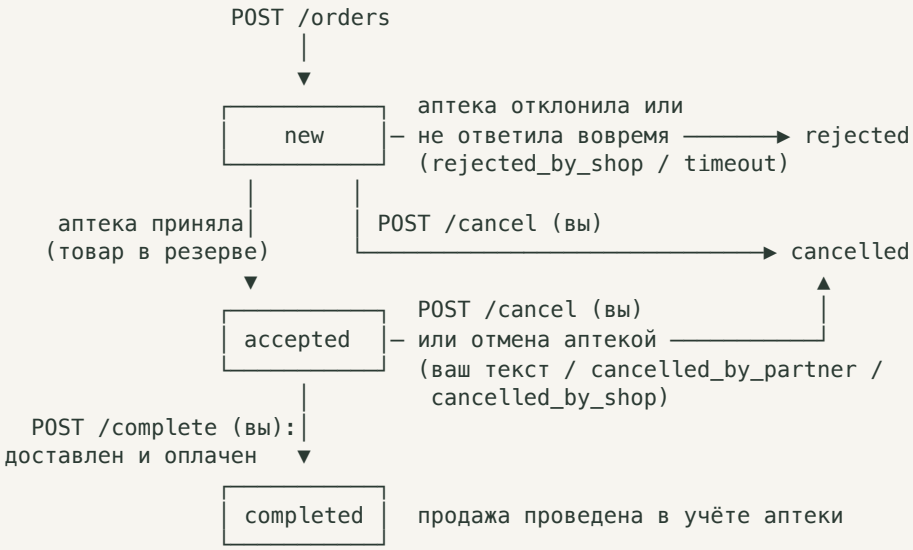
Ответ на создание (и на любой запрос заказа) — заказ целиком:

JSON

```
{
  "id": 1042,
  "order_id": "A-100045",
  "status": "new",
  "reason": null,
  "test": true,
  "items": [
    { "product_id": 701, "name": "Витамин D3 2000 МЕ, 60 капсул", "quantity": 2, "price": 145000,
"total": 290000 },
    { "product_id": 705, "name": "Омега-3, 90 капсул", "quantity": 1, "price": 210000, "total":
210000 }
  ],
  "total": 500000,
  "customer": { "name": "Тестовый покупатель", "phone": "+998900000000" },
  "delivery": { "type": "courier", "address": "Ташкент, тестовый адрес, 1", "comment": "Домофон
00" },
  "comment": "Позвонить за 15 минут",
  "decision_deadline": "2026-11-12T08:05:00.000Z",
  "created_at": "2026-11-12T08:00:00.000Z",
  "updated_at": "2026-11-12T08:00:00.000Z",
  "history": [
    { "status": "new", "at": "2026-11-12T08:00:00.000Z", "reason": null }
  ]
}
```

Поле	Что это
`id`	Номер заказа у нас — назовите его, если звоните в аптеку.
`order_id`	Ваш номер.
`status`	`new`, `accepted`, `rejected`, `cancelled`, `completed` (раздел 9.2).
`reason`	Причина для `rejected` / `cancelled`, иначе `null`.
`test`	`true` — тестовый заказ, склад и учёт не затронуты.
`items[].price`, `items[].total`, `total`	Цена за штуку, сумма строки и сумма заказа в сумах — по нашему каталогу.
`decision_deadline`	До какого времени аптека ответит. Есть только у статуса `new`, у остальных — `null`.
`history`	Все смены статуса по порядку: `{status, at, reason}`.

9.2 Жизненный цикл заказа



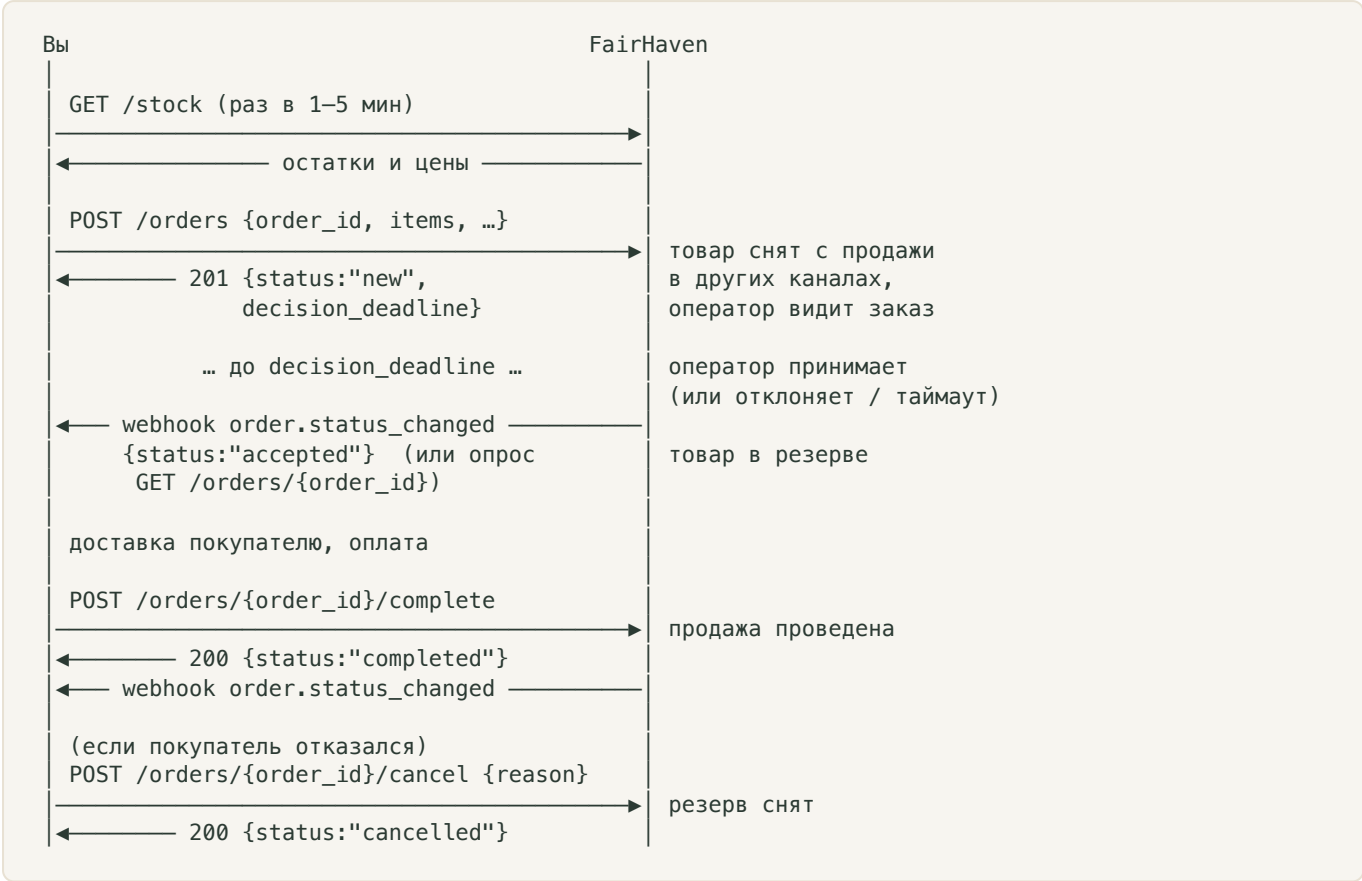
Статус	Что значит	Что с товаром (режим `live`)	Ваши действия
`new`	Заказ получен, аптека решает.	Сразу снят с продажи в других каналах.	Ждать решения; можно отменить.
`accepted`	Аптека приняла.	Зарезервирован в учёте аптеки.	Доставить; затем `complete` или `cancel`.
`rejected`	Аптека отклонила или не ответила вовремя.	Возвращён в продажу.	Сообщить покупателю. Финальный статус.
`cancelled`	Отменён вами или аптекой.	Возвращён в продажу.	Финальный статус.
`completed`	Вы подтвердили доставку и оплату.	Продан.	Финальный статус.

Значения `reason`:

`reason`	Статус	Когда
`rejected_by_shop`	`rejected`	Оператор аптеки отклонил заказ (в тесте — робот по `test_outcome: "reject"`).
`timeout`	`rejected`	Аптека не ответила до `decision_deadline` (только `live`).
`cancelled_by_partner`	`cancelled`	Вы отменили заказ без указания причины.
ваш текст	`cancelled`	Вы отменили заказ и передали `reason`.
`cancelled_by_shop`	`cancelled`	Аптека отменила заказ со своей стороны.

Окно решения. Аптека отвечает на заказ в течение окна решения — по умолчанию 5 минут (точное время каждого заказа — в `decision_deadline`). Если оператор не успел, заказ автоматически становится `rejected` с причиной `timeout`, а товар возвращается в продажу. В тестовом режиме решение принимает робот через ~10–15 секунд.

9.3 Полная последовательность



Каждая смена статуса — в том числе вызванная вашим же `cancel` или `complete` — приходит и webhook-событием. Это нормально: обрабатывайте событие идемпотентно (раздел 10).

9.4 Получение заказа: `GET /orders/{order_id}`

```bash
curl -s "$FH/orders/A-100045" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

По **вашему** `order_id`. Подходит любой ключ. Нет такого заказа — `404 order_not_found`.

9.5 Список заказов: `GET /orders`

```bash
curl -s "$FH/orders?status=accepted&limit=100&offset=0" -H "Authorization: Bearer $FAIRHAVEN_TOKEN"
```

Новые сверху. Фильтр `status` — один из `new`, `accepted`, `rejected`, `cancelled`, `completed` (другое значение — `422 validation_error`). Удобно для сверки в конце дня и для восстановления после сбоя на вашей стороне.

Список показывает заказы текущего режима: в `test` — тестовые, в `live` — боевые. Тестовые заказы после перехода в `live` через API не видны.

9.6 Отмена: `POST /orders/{order_id}/cancel` (секрет заказов)

```bash
curl -s -X POST "$FH/orders/A-100045/cancel" \
  -H "Authorization: Bearer $FAIRHAVEN_SECRET" \
  -H "Content-Type: application/json" \
  -d '{"reason": "Покупатель отказался"}'
```

Можно из статусов `new` и `accepted`. Тело необязательно; `reason` — до 200 символов, попадёт в поле `reason` заказа (без него — `cancelled_by_partner`). Резерв снимается, товар возвращается в продажу. Из других статусов — `409 invalid_status` (в `details.status` — текущий статус).

9.7 Завершение: `POST /orders/{order_id}/complete` (секрет заказов)

```bash
curl -s -X POST "$FH/orders/A-100045/complete" -H "Authorization: Bearer $FAIRHAVEN_SECRET"
```

Вызывайте, когда заказ **доставлен и оплачен** покупателем. Только из статуса `accepted`. В этот момент продажа проводится в учёте аптеки, поэтому:

- не вызывайте `complete` заранее «на всякий случай» — отменить проведённую продажу через API нельзя;
- если покупатель отказался — вызывайте `cancel`, а не `complete`;
- повторный `complete` ничего не меняет и отвечает `409 invalid_status` с `details.status: "completed"` — считайте это успехом;
- при `503 billz_unavailable` повторите через минуту;
- при `503 sale_pending_review` **не повторяйте сразу**: продажу проверяет аптека. Когда она подтвердит продажу, заказ сам станет `completed` (придёт webhook). Если через 15 минут статус всё ещё `accepted` — повторите `complete`, это безопасно.

10. Ошибки

HTTP	error.code	Что значит	Что делать
400	`invalid_json`	Тело запроса — не JSON.	Исправьте формирование тела и `Content-Type`.
401	`unauthorized`	Ключ не передан, неверен, отозван, или на метод заказов отправлен токен чтения.	Проверьте заголовок и какой ключ используется. Не повторяйте в цикле.
403	`partner_disabled`	Доступ партнёра выключен (режим `off`).	Свяжитесь с FairHaven.
404	`product_not_found`	Товара нет в каталоге (`details.product_id`).	Обновите каталог, уберите товар с витрины.

HTTP	error.code	Что значит	Что делать
404	`order_not_found`	Нет заказа с таким `order_id` у вас.	Проверьте `order_id`.
404	`not_found`	Нет такого адреса API.	Проверьте путь и метод.
409	`insufficient_stock`	Не хватает остатка: `details.items[] = {product_id, requested, available}`.	Предложите покупателю доступное количество и отправьте заказ заново с **новым** `order_id` (или с тем же, см. FAQ).
409	`invalid_status`	Действие недоступно в текущем статусе (`details.status`).	Получите заказ и действуйте по его статусу.
409	`order_busy`	Заказ прямо сейчас меняется другим действием.	Повторите через несколько секунд.
409	`webhook_not_configured`	`POST /webhook/test`, а адрес webhook ещё не задан.	Сообщите нам адрес.
413	`payload_too_large` (ответ может быть не в JSON)	Тело больше 256 КБ.	Уменьшите запрос.
422	`validation_error`	Ошибка в данных; `message` говорит, в каком поле.	Исправьте запрос. Повтор без изменений не поможет.
429	— (тело `{"detail": …}` или `{"error": "rate_limited"}`)	Превышен лимит запросов (на ключ или на IP).	Подождите `Retry-After` секунд (если заголовка нет — 1–5 секунд), снизьте частоту.
500	`internal_error`	Внутренняя ошибка.	Повторите позже; если повторяется — напишите нам.
503	`billz_unavailable`	Учётная система аптеки не ответила.	Повторите через минуту.
503	`billz_writes_disabled`	Проведение продаж временно выключено.	Повторите позже.
503	`release_failed`	Не удалось снять резерв при отмене.	Повторите через минуту.
503	`sale_pending_review`	Продажа в учёте под проверкой аптеки.	**Не повторяйте сразу.** После подтверждения заказ сам станет `completed`; если через 15 минут он всё ещё `accepted` — повторите `complete`.

Сетевые ошибки и таймауты при `POST /orders` безопасно повторять с тем же `order_id` — второй заказ не создастся.

11. Webhooks — уведомления о смене статуса

Чтобы не опрашивать заказы, сообщите нам адрес `https://…` (только HTTPS, без логина и пароля в адресе, до 500 символов). Мы пришлём **секрет подписи** `whsec_…` — им вы проверяете, что запрос пришёл от нас.

11.1 Что приходит

`POST` на ваш адрес, `Content-Type: application/json`, `User-Agent: FairHaven-Webhooks/1.0`:

```
X-FairHaven-Event: order.status_changed
X-FairHaven-Event-Id: 6733a1f0c2e4b5a6d7e8f901
X-FairHaven-Timestamp: 1762934400
X-FairHaven-Signature: v1=5d41402abc4b2a76b9719d911017c592…
```

JSON

```
{
  "id": "6733a1f0c2e4b5a6d7e8f901",
  "type": "order.status_changed",
  "created_at": "2026-11-12T08:02:10.000Z",
  "data": { "id": 1042, "order_id": "A-100045", "status": "accepted", "…": "заказ целиком, как в
GET /orders/{order_id}" }
}
```

Поле / заголовок	Что это
`id`, `X-FairHaven-Event-Id`	Уникальный id события. Повторная доставка приходит с тем же id.
`type`, `X-FairHaven-Event`	`order.status_changed` или `ping` (проверка).
`created_at`	Время, когда произошло событие. Одинаково во всех повторных доставках.
`data`	Для `order.status_changed` — заказ целиком в состоянии на момент события. Для `ping` — `{"message": "FairHaven webhook test"}`.
`X-FairHaven-Timestamp`	Время отправки, Unix-секунды.
`X-FairHaven-Signature`	`v1=` + hex HMAC-SHA256.

События приходят по каждой смене статуса: `accepted`, `rejected`, `cancelled`, `completed`. Создание заказа событием не сопровождается — его результат вы получаете в ответе на `POST /orders`. Тестовые заказы тоже присылают события (с `data.test: true`).

11.2 Как отвечать

- Ответьте любым кодом `2xx` **быстро** (до 10 секунд), тело ответа не читается. Тяжёлую обработку делайте после ответа, в очереди.
- Любой другой код, перенаправление (`3xx` не выполняется), таймаут 10 секунд или сетевая ошибка — неудача. Повторы: через 10 с, 1 мин, 5 мин, 15 мин, 1 ч, 3 ч, 6 ч, 12 ч; после этого событие больше не отправляется (заказ по-прежнему доступен через `GET /orders/{order_id}`).

- Доставка «как минимум один раз»: одно событие может прийти дважды. Запоминайте обработанные `id` и пропускайте повторы.
- Порядок сохраняется: пока более старое событие не доставлено, более новые ваших заказов ждут его. Новый статус не обгонит старый.
- События, возникшие, пока адрес webhook не был задан, позже не досылаются — сверьтесь через `GET /orders`.

11.3 Проверка подписи

Подпись считается от **сырого тела запроса** (байты как пришли, до разбора JSON):

```
signature = hex( HMAC-SHA256( webhook_secret, "<X-FairHaven-Timestamp>.<сырое тело>" ) )
X-FairHaven-Signature: v1=<signature>
```

Проверьте подпись сравнением за постоянное время и отклоняйте запросы со временем старше ~5 минут (защита от повтора перехваченного запроса). Каждая повторная попытка подписывается заново, со свежим временем.

Python (Flask):

PYTHON

```python
import hashlib, hmac, os, time
from flask import Flask, request, abort

app = Flask(__name__)
WEBHOOK_SECRET = os.environ["FAIRHAVEN_WEBHOOK_SECRET"]  # whsec_…
seen_ids = set()  # в продакшене — таблица в БД с уникальным ключом


def verify(secret: str, timestamp: str, raw_body: bytes, signature: str, max_age: int = 300) ->
bool:
    try:
        ts = int(timestamp)
    except (TypeError, ValueError):
        return False
    if abs(time.time() - ts) > max_age:
        return False
    expected = hmac.new(secret.encode(), timestamp.encode() + b"." + raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(f"v1={expected}", signature or "")


@app.post("/fairhaven/webhook")
def fairhaven_webhook():
    raw = request.get_data()  # сырые байты, до разбора JSON
    if not verify(WEBHOOK_SECRET,
                  request.headers.get("X-FairHaven-Timestamp", ""),
                  raw,
                  request.headers.get("X-FairHaven-Signature", "")):
        abort(401)
    event = request.get_json()
    if event["id"] in seen_ids:
        return "", 204  # повтор — уже обработали
    seen_ids.add(event["id"])
    if event["type"] == "order.status_changed":
        order = event["data"]
        # поставьте в очередь: обновить заказ order["order_id"] → order["status"]
    return "", 204
```

Node.js (Express):

```JS
const crypto = require('crypto');
const express = require('express');

const app = express();
const WEBHOOK_SECRET = process.env.FAIRHAVEN_WEBHOOK_SECRET; // whsec_…
const seenIds = new Set(); // в продакшене — таблица в БД с уникальным ключом

function verify(secret, timestamp, rawBody, signature, maxAge = 300) {
  const ts = Number(timestamp);
  if (!Number.isInteger(ts) || Math.abs(Date.now() / 1000 - ts) > maxAge) return false;
  const expected = 'v1=' + crypto.createHmac('sha256', secret)
    .update(`${timestamp}.`).update(rawBody).digest('hex');
  const a = Buffer.from(expected);
  const b = Buffer.from(String(signature || ''));
  return a.length === b.length && crypto.timingSafeEqual(a, b);
}

// express.raw — чтобы получить тело байтами, а не уже разобранный JSON
app.post('/fairhaven/webhook', express.raw({ type: 'application/json' }), (req, res) => {
  if (!verify(WEBHOOK_SECRET, req.get('X-FairHaven-Timestamp'), req.body, req.get('X-FairHaven-
Signature'))) {
    return res.sendStatus(401);
  }
  const event = JSON.parse(req.body.toString('utf8'));
  if (seenIds.has(event.id)) return res.sendStatus(204);
  seenIds.add(event.id);
  if (event.type === 'order.status_changed') {
    const order = event.data;
    // поставьте в очередь: обновить заказ order.order_id → order.status
  }
  return res.sendStatus(204);
});

app.listen(3000);
```

Частая ошибка — считать подпись от заново сериализованного JSON (`JSON.stringify(req.body)`, `json.dumps(...)`): порядок ключей и пробелы могут отличаться, подпись не сойдётся. Берите сырое тело.

11.4 Проверка адреса: `POST /webhook/test` (секрет заказов)

```BASH
curl -s -X POST "$FH/webhook/test" -H "Authorization: Bearer $FAIRHAVEN_SECRET"
```

Мы сразу отправим на ваш адрес событие `ping` (подписанное так же, как настоящие) и вернём результат:

```JSON
{ "delivered": true, "status_code": 204, "error": null }
```

`delivered: false` — смотрите `status_code` (что ответил ваш сервер) или `error` (`timeout`, `network_error`). Ping отправляется один раз, без повторов.

12. Без webhook: опрос

Webhook не обязателен. Без него:

1. После `POST /orders` опрашивайте `GET /orders/{order_id}` раз в 10–30 секунд, пока статус `new`. Решение придёт не позже `decision_deadline` (плюс несколько секунд).
2. Заказы в `accepted` можно проверять реже — раз в несколько минут: аптека может отменить заказ (`cancelled_by_shop`) или завершённая продажа может выйти из проверки (`sale_pending_review`).
3. Для сверки — `GET /orders?status=new` и `GET /orders?status=accepted`.

Даже с webhook полезно раз в 10–15 минут сверять открытые заказы опросом — на случай, если ваш сервер был недоступен дольше суток.

13. Тестовый режим: что проверить

В режиме `test` пройдите эти сценарии. Используйте для тестовых заказов `order_id` с отдельным префиксом (например, `TEST-…`) — так их проще отличать в логах. Тестовые и боевые заказы ведутся раздельно: номер, использованный в тесте, после перехода в `live` создаёт новый боевой заказ, а `GET /orders/{order_id}` показывает заказ того режима, в котором партнёр сейчас работает.

№	Сценарий	Ожидаемый результат
1	`GET /me` токеном и секретом	`200`, `mode: "test"`, `key` — `token` / `secret`.
2	Запрос без ключа и с неверным ключом	`401 unauthorized`.
3	`POST /orders` токеном чтения	`401 unauthorized` («needs the order secret»).
4	Полная выгрузка `/products` и `/stock` постранично	Число строк = `total`; товары с `quantity: 0` на витрине помечены «нет в наличии».
5	Заказ → ждать → `accepted` → `complete`	`201 new` → `accepted` (~10–15 с) → `200 completed`.
6	Заказ с `"test_outcome": "reject"`	`rejected`, `reason: "rejected_by_shop"`; покупателю показан отказ.
7	Заказ → `cancel` пока `new`	`cancelled`, `reason` — ваш текст или `cancelled_by_partner`.
8	Заказ → `accepted` → `cancel`	`cancelled`.
9	Повтор `POST /orders` с тем же `order_id`	`200`, тот же `id`, второй заказ не создан.
10	Заказ на количество больше `quantity` из `/stock`	`409 insufficient_stock`, в `details.items` — сколько есть.
11	Заказ с несуществующим `product_id`	`404 product_not_found`.
12	Заказ без `items` / с `quantity: 0` / без `order_id`	`422 validation_error`.

№	Сценарий	Ожидаемый результат
13	`complete` для заказа в `new` или `rejected`	`409 invalid_status`.
14	Повторный `complete`	`409 invalid_status`, `details.status: "completed"` — ваш код считает это успехом.
15	`POST /webhook/test`	`delivered: true`; ваш сервер принял ping и проверил подпись.
16	Webhook с испорченной подписью или старым временем (подделайте сами)	Ваш сервер отвечает отказом и не меняет заказ.
17	Один и тот же webhook дважды	Второй раз ничего не меняется.
18	Ваш webhook временно отвечает `500`	Событие приходит повторно через 10 с, 1 мин…
19	Ответ `429` (имитация на вашей стороне)	Клиент ждёт `Retry-After` и продолжает.

В тестовом режиме остатки настоящие, поэтому сценарий 10 проверяется на реальных числах, но заказы их не уменьшают.

14. Переход в боевой режим

Перед переключением в `live`:

- [ ] Все сценарии раздела 13 пройдены.
- [ ] Ключи лежат в переменных окружения / хранилище секретов, не в коде и не в логах; секрет заказов есть только у сервиса оформления заказов.
- [ ] Витрина берёт цены и количество из `/stock` и обновляет их не реже раза в 5 минут; товары, пропавшие из `/stock`, снимаются с продажи.
- [ ] `order_id` боевых заказов уникальны и не пересекаются с тестовыми.
- [ ] Покупатель видит отказ, если заказ `rejected` (в том числе по `timeout`).
- [ ] `complete` вызывается только после доставки и оплаты; отказ покупателя ведёт к `cancel`.
- [ ] Webhook проверяет подпись и время, отвечает быстро, пропускает повторы — или настроен опрос по разделу 12.
- [ ] Есть контакт ответственного с вашей стороны на случай проблем.

Затем согласуем с вами время, вместе проведём первый реальный заказ и переключим режим. Ключи не меняются — меняется только `partner.mode` в `GET /me`.

15. Частые вопросы

Товар закончился между запросом `/stock` и заказом. Что будет? Заказ не создастся: ответ `409 insufficient_stock`, в `details.items` — по каждому товару `requested` (сколько просили) и `available` (сколько есть сейчас). Ни одна строка заказа не резервируется. Предложите покупателю доступное количество и отправьте заказ заново.

Можно ли повторно отправить заказ, если ответ не дошёл? Да, это безопасно: отправьте его с тем же `order_id`. Если заказ уже создан, придёт `200` с сохранённым заказом, второго не будет. Учтите: при повторе возвращается **первоначальный** заказ, новое тело запроса не применяется. Чтобы оформить другой состав, используйте новый `order_id`. Если первая попытка закончилась ошибкой (`409`, `422`, `404`), заказ не создан, и тот же `order_id` можно использовать снова.

Как узнать решение аптеки? Webhook `order.status_changed` (раздел 11) или опрос `GET /orders/{order_id}` (раздел 12). Решение придёт не позже `decision_deadline`: `accepted` — принят, `rejected` — отклонён (`reason` покажет, отклонил оператор или истекло время).

Можно ли изменить заказ (состав, адрес)? Нет. Отмените заказ (`cancel`) и отправьте новый с новым `order_id`.

Сколько держится резерв по принятому заказу? Пока вы не вызовете `complete` или `cancel` (или аптека не отменит заказ). Пожалуйста, закрывайте заказы сразу после доставки или отказа — пока резерв висит, товар недоступен другим покупателям.

Почему цена в заказе отличается от цены на моей витрине? Заказ всегда считается по текущей цене нашего каталога. Если цена изменилась после вашего последнего `/stock`, в ответе на `POST /orders` будет новая цена. Обновляйте `/stock` чаще или сверяйте `total` ответа со своим и при расхождении отменяйте заказ.

Самовывоз поддерживается? Да: `"delivery": {"type": "pickup"}`. Адрес можно не указывать.

Куда деваются тестовые заказы после перехода в боевой режим? Они остаются в нашей истории, но API в режиме `live` показывает только боевые заказы. Тестовые заказы склад и учёт никогда не трогали.

Пришёл `503 sale_pending_review` на `complete`. Что делать? Не повторять сразу. Учётная система ответила неоднозначно, и продажу проверяет сотрудник аптеки. Когда он подтвердит продажу, заказ сам станет `completed` и придёт webhook. Если через 15 минут заказ всё ещё `accepted` — повторите `complete`: это безопасно, продажа не задвоится.

Нужно ли подтверждать webhook телом ответа? Нет, важен только код `2xx`. Тело ответа не читается.

Как получить спецификацию для Postman / Swagger?
`https://api.fairhaven.uz/partner/openapi.yaml` — без ключа.